

DSA MCQ's

UNIT-1 MCQs

Problem Solving & Computational Thinking (PG-DAC / CCEE Focus)

1. The primary objective of problem definition is to:

- A. Write an efficient algorithm B. Identify the programming language C. Clearly understand what needs to be solved D. Optimize memory usage

Answer: C

2. Which of the following is NOT a part of problem definition?

- A. Inputs B. Outputs C. Constraints D. Programming syntax

Answer: D

3. Constraints in a problem mainly affect:

- A. Code readability B. Algorithm selection C. User interface design D. Documentation style

Answer: B

4. A problem statement that lacks clarity usually results in:

- A. Faster implementation B. Optimal solutions C. Incorrect or incomplete solutions D. Reduced testing effort

Answer: C

5. Identifying the *real problem* rather than the *perceived problem* helps to:

- A. Reduce compilation errors B. Avoid solving irrelevant issues C. Improve syntax accuracy D. Increase code length

Answer: B

6. Which activity best helps distinguish the real problem from the perceived problem?

- A. Debugging B. Root cause analysis C. Coding D. Compilation

Answer: B

7. Breaking a complex problem into smaller parts is known as:

- A. Abstraction B. Decomposition C. Pattern matching D. Optimization

Answer: B

8. Reusable components mainly improve:

- A. Memory consumption B. Development efficiency and reliability C. User interaction D. Input validation

Answer: B

9. Trial and Error approach is best suited for:

- A. Large-scale systems B. Time-critical applications C. Small and simple problems D. Real-time systems

Answer: C

10. Divide and Conquer strategy works by:

A. Solving the problem directly B. Breaking the problem into independent sub-problems C. Using random inputs D. Avoiding recursion

Answer: B

11. Which of the following algorithms uses Divide and Conquer?

A. Linear search B. Bubble sort C. Merge sort D. Insertion sort

Answer: C

12. Pattern recognition is MOST useful when:

A. The problem is entirely new B. Similar problems have been solved earlier C. Constraints are unknown D. Inputs are random

Answer: B

13. Abstraction mainly helps in:

A. Writing longer code B. Ignoring unnecessary details C. Increasing hardware dependency D. Eliminating algorithms

Answer: B

14. Algorithmic thinking ensures that a solution is:

A. Random and flexible B. Ambiguous C. Step-wise and deterministic D. Hardware dependent

Answer: C

SECTION C: Steps in Problem Solving

15. Which is the correct order of problem-solving steps?

A. Implement → Design → Analyze B. Define → Analyze → Design → Implement C. Analyze → Define → Implement D. Design → Define → Test

Answer: B

16. Selecting data structures is done AFTER:

A. Implementation B. Optimization

C. Algorithm design D. Testing

Answer: C

17. The purpose of problem analysis is to:

A. Write code B. Identify edge cases and relationships C. Optimize memory D. Choose programming language

Answer: B

18. Which step ensures correctness of the solution?

A. Implementation B. Optimization C. Testing and validation D. Algorithm design

Answer: C

19. Optimization primarily focuses on:

A. Removing comments B. Improving time and space efficiency C. Enhancing UI D. Increasing lines of code

Answer: B

20. Boundary value testing is performed during:

A. Problem definition B. Algorithm design C. Testing phase D. Optimization phase

Answer: C

21. Computational thinking is best described as:

A. Writing computer programs B. Thinking like a computer C. Structured problem-solving approach D. Learning programming languages

Answer: C

22. Decomposition in computational thinking means:

A. Removing constraints B. Dividing a problem into manageable parts C. Identifying patterns D. Writing algorithms

Answer: B

23. Pattern matching in computational thinking helps to:

A. Avoid abstraction B. Identify similarities across problems C. Increase redundancy D. Reduce decomposition

Answer: B

24. Which of the following is an example of abstraction?

A. Listing all variables B. Ignoring irrelevant details C. Writing loops D. Debugging

Answer: B

25. Algorithm design in computational thinking requires:

A. Random steps B. Ambiguous instructions C. Finite and well-defined steps D. Hardware-specific instructions

Answer: C

26. Which property is NOT mandatory for an algorithm?

A. Finite B. Deterministic C. Ambiguous D. Effective

Answer: C

27. Choosing an inappropriate data structure mainly impacts:

A. Output correctness B. Performance and efficiency C. Input validation D. Compilation

Answer: B

28. A problem with multiple constraints generally requires:

A. Trial and error B. Brute force solution C. Careful algorithm selection D. No testing

Answer: C

29. Which thinking skill is MOST useful before coding?

A. Syntax memorization B. Debugging C. Problem analysis D. Compilation

Answer: C

30. Which phase consumes the MOST time in real-world software development?

A. Coding B. Testing C. Requirement analysis D. Compilation

Answer: C

31. Failure to identify constraints early may lead to:

A. Efficient algorithms B. Incorrect optimization C. Faster execution D. Simpler design

Answer: B

32. Computational thinking is applicable:

A. Only in programming B. Only in mathematics
C. In real-life problem solving also D. Only in algorithms

Answer: C

33. A well-defined problem always has:

A. Multiple ambiguous outputs B. Clear inputs and outputs C. No constraints D. No assumptions

Answer: B

34. Modular problem solving improves:

A. Code redundancy B. Maintainability and scalability C. Compilation time D. Memory leaks

Answer: B

35. Which is the MOST critical step before implementation?

A. Optimization B. Testing C. Algorithm design D. Debugging

Answer: C

UNIT-2 MCQs

Algorithms & Fundamentals (CCEE / CDAC PG-DAC Focus)

1. An algorithm is best defined as:

A. A computer program written in any language B. A finite sequence of well-defined steps to solve a problem C. A flowchart representation of a solution D. A mathematical equation

Answer: B

2. Which of the following is NOT a mandatory characteristic of an algorithm?

A. Input B. Output C. Ambiguity D. Finiteness

Answer: C

3. An algorithm that does not terminate violates which property?

A. Definiteness B. Effectiveness C. Finiteness D. Input

Answer: C

4. Which characteristic ensures that each step of an algorithm is clear and unambiguous?

A. Effectiveness B. Definiteness C. Output D. Input

Answer: B

5. An algorithm that produces no output is considered:

A. Incomplete B. Incorrect C. Invalid D. Inefficient

Answer: C

6. Which property of an algorithm ensures that all steps are practically executable?

A. Finiteness B. Definiteness C. Effectiveness D. Output

Answer: C

7. Zero input algorithms are:

A. Invalid B. Valid if output exists C. Always infinite D. Non-deterministic

Answer: B

8. Pseudocode is:

A. A programming language B. Machine-dependent code C. Language-independent algorithm description D. Compiler-specific syntax

Answer: C

9. The main purpose of pseudocode is to:

A. Replace programming languages B. Improve execution speed C. Express algorithm logic clearly D. Reduce memory usage

Answer: C

10. Which representation uses graphical symbols to depict control flow?

A. Pseudocode B. Structured English C. Flowchart D. Source code

Answer: C

11. Which flowchart symbol represents decision making?

A. Rectangle B. Oval C. Parallelogram D. Diamond

Answer: D

12. Structured English is MOST useful for:

A. Compiler design B. Hardware description C. Requirement documentation D. Machine language programming

Answer: C

13. Which algorithm representation becomes inefficient for very large problems?

A. Pseudocode B. Structured English C. Flowcharts D. Source code

Answer: C

14. Default execution order in an algorithm is:

A. Conditional B. Recursive C. Iterative D. Sequential

Answer: D

15. Conditional branching allows:

A. Repetition of statements B. Selection of execution path C. Infinite execution D. Memory optimization

Answer: B

16. Which construct is MOST suitable when the number of iterations is known in advance?

A. WHILE loop B. DO-WHILE loop

C. FOR loop D. Recursion

Answer: C

17. A loop that executes at least once is:

A. FOR loop B. WHILE loop C. DO-WHILE loop D. Recursive loop

Answer: C

18. Recursion requires which of the following to avoid infinite calls?

A. Loop counter B. Exit statement C. Base case D. Global variable

Answer: C

19. Recursive solutions typically consume more:

A. CPU time only B. Stack memory C. Heap memory D. Cache memory

Answer: B

20. Which construct improves clarity by eliminating repetitive code?

A. Conditional B. Recursion C. Sequential D. Input

Answer: B

21. The main objective of algorithm analysis is to:

A. Improve code readability B. Measure algorithm efficiency C. Reduce syntax errors D. Select programming language

Answer: B

22. Time complexity measures:

A. Memory usage B. Execution time as input grows C. Number of variables D. CPU speed

Answer: B

23. Space complexity accounts for:

A. Input data only B. Auxiliary memory used C. Output only D. CPU registers only

Answer: B

24. An algorithm with nested loops generally has:

A. Constant time complexity B. Linear time complexity C. Higher time complexity D. Logarithmic complexity

Answer: C

25. Big-O notation is used to express:

A. Exact execution time B. Worst-case performance C. Best-case performance only D. Memory address

Answer: B

26. Which complexity grows fastest?

A. $O(\log n)$ B. $O(n)$ C. $O(n^2)$ D. $O(1)$

Answer: C

27. Recursive algorithms often trade:

A. Accuracy for speed B. Memory for simplicity C. Time for correctness D. Input for output

Answer: B

28. A well-designed algorithm primarily ensures:

A. Short code B. Hardware independence C. Correctness and efficiency D. Easy compilation

Answer: C

29. Choosing a poor algorithm mainly affects:

A. Syntax B. Output format C. Performance D. Input method

Answer: C

30. Algorithm analysis should be performed:

A. After compilation B. Before implementation C. During execution D. After deployment

Answer: B

31. Which algorithm construct is closely associated with Divide and Conquer?

A. Sequential B. Iteration C. Recursion D. Conditional

Answer: C

32. Which representation is MOST suitable for exam answers?

A. Source code B. Assembly language

C. Pseudocode D. Machine code

Answer: C

33. Space-time trade-off means:

A. Less time always needs less memory B. Faster algorithms may use more memory C. Memory and time are unrelated D. Memory optimization increases time always

Answer: B

34. Which property ensures an algorithm does not run forever?

A. Definiteness B. Input C. Effectiveness D. Finiteness

Answer: D

35. The MOST critical factor while selecting an algorithm is:

A. Programming language B. Input size and constraints C. Developer preference D. Coding style

Answer: B

UNIT 3 – COMPLEXITY ANALYSIS

High-Level MCQs for CCEE (CDAC)

1. Time complexity of an algorithm refers to:

A. Actual execution time on a machine B. Number of instructions in source code C. Growth of execution time with input size D. CPU clock speed

Answer: C

2. Which case is MOST commonly considered in complexity analysis?

A. Best case B. Average case C. Worst case D. Random case

Answer: C

3. Worst-case time complexity is preferred because it:

A. Is easier to calculate B. Represents average performance C. Guarantees an upper bound D. Minimizes execution time

Answer: C

4. An algorithm whose execution time does not depend on input size has:

A. $O(n)$ complexity B. $O(\log n)$ complexity C. $O(1)$ complexity D. $O(n^2)$ complexity

Answer: C

5. Best-case complexity of linear search occurs when:

A. Element is not present B. Element is at the last position C. Element is at the first position D. Array is sorted

Answer: C

6. Big-O notation represents:

A. Lower bound B. Exact bound C. Upper bound D. Average bound

Answer: C

7. Ω (Omega) notation defines:

A. Maximum execution time B. Minimum execution time C. Exact execution time D. Worst-case execution time

Answer: B

8. Θ (Theta) notation indicates:

A. Only best case B. Only worst case C. Tight bound D. Upper bound

Answer: C

9. If an algorithm has time complexity $O(n^2)$ and $\Omega(n^2)$, its Θ complexity is:

A. $\Theta(n)$ B. $\Theta(n \log n)$ C. $\Theta(n^2)$ D. Cannot be determined

Answer: C

10. Which notation is MOST useful for comparing algorithms?

A. Ω B. Θ C. O D. Constant notation

Answer: C

11. Which grows the fastest as n increases?

A. $O(n \log n)$ B. $O(n^2)$

C. $O(2^n)$ D. $O(\log n)$

Answer: C

12. Which is the MOST efficient for very large input sizes?

A. $O(n^2)$ B. $O(n \log n)$ C. $O(\log n)$ D. $O(2^n)$

Answer: C

13. Which complexity is considered optimal for comparison-based sorting?

A. $O(n)$ B. $O(n \log n)$ C. $O(n^2)$ D. $O(\log n)$

Answer: B

14. Exponential time algorithms are generally:

A. Highly scalable B. Memory efficient C. Impractical for large inputs D. Faster than polynomial time

Answer: C

15. $O(n!)$ complexity is typically associated with:

A. Searching B. Sorting C. Permutations D. Traversal

Answer: C

16. Time complexity of:

for $i = 1$ to n

print(i)

A. $O(1)$ B. $O(\log n)$

C. $O(n)$ D. $O(n^2)$

Answer: C

17. Time complexity of:

for $i = 1$ to n

for $j = 1$ to n

A. $O(n)$ B. $O(n \log n)$ C. $O(n^2)$ D. $O(2n)$

Answer: C

18. Time complexity of:

for i = 1 to n

for j = 1 to i

A. $O(n)$ B. $O(n \log n)$ C. $O(n^2)$ D. $O(\log n)$

Answer: C

19. Time complexity of:

for i = 1 to n

i = i * 2

A. $O(n)$ B. $O(n \log n)$ C. $O(\log n)$ D. $O(1)$

Answer: C

20. Time complexity of:

for i = 1 to n

for j = 1 to $\log n$

A. $O(\log n)$ B. $O(n)$ C. $O(n \log n)$ D. $O(n^2)$

Answer: C

21. Which factor MOST affects recursive space complexity?

A. Number of parameters B. Stack depth C. Input size D. Return type

Answer: B

22. Time complexity of:

$T(n) = T(n - 1) + c$

A. $O(1)$ B. $O(\log n)$ C. $O(n)$ D. $O(n^2)$

Answer: C

23. Time complexity of:

$T(n) = T(n / 2) + c$

A. $O(n)$ B. $O(\log n)$ C. $O(n \log n)$ D. $O(2^n)$

Answer: B

24. Time complexity of:

$T(n) = 2T(n / 2) + n$

A. $O(n)$ B. $O(\log n)$ C. $O(n \log n)$ D. $O(n^2)$

Answer: C

25. Recursive Fibonacci algorithm has time complexity:

A. $O(n)$ B. $O(n \log n)$ C. $O(2^n)$ D. $O(n^2)$

Answer: C

26. Stack space complexity of factorial recursion is:

A. $O(1)$ B. $O(\log n)$ C. $O(n)$ D. $O(n^2)$

Answer: C

27. Which statement is TRUE?

A. Average case is always worst case B. Worst case is always best case C. Worst case provides performance guarantee D. Best case is most realistic

Answer: C

28. Ignoring constants means:

A. $O(2n) \neq O(n)$ B. $O(n + 100) = O(n)$ C. $O(n^2 + n) = O(n)$ D. $O(n \log n) = O(n^2)$

Answer: B

29. If an algorithm has nested loops but inner loop runs constant times, complexity is:

A. $O(n^2)$ B. $O(n \log n)$ C. $O(n)$ D. $O(1)$

Answer: C

30. Which algorithm characteristic MOST impacts scalability?

A. Syntax B. Memory address C. Time complexity D. Variable naming

Answer: C

31. Recursive solutions are often:

A. Faster than iterative B. Memory efficient C. Elegant but memory expensive D. Always optimal

Answer: C

32. For large input sizes, which should be avoided?

A. $O(\log n)$ B. $O(n)$ C. $O(n \log n)$ D. $O(2^n)$

Answer: D

33. Which complexity dominates when multiple terms exist?

A. Lowest order B. Highest order C. Constant term D. Random term

Answer: B

34. Time complexity analysis is independent of:

A. Input size B. Hardware C. Algorithm design D. Loop structure

Answer: B

35. In CCEE, complexity questions MOSTLY test:

A. Coding syntax B. Mathematical proofs C. Growth rate analysis D. Output formatting

Answer: C

UNIT 4 & UNIT 5

High-Level MCQs for CCEE (CDAC PG-DAC)

1. An Abstract Data Type (ADT) primarily focuses on:

A. Memory allocation strategy B. Programming language syntax C. Logical behavior and operations D. Hardware architecture

Answer: C

2. Which of the following BEST describes an ADT?

A. A concrete data structure stored in memory B. A logical model defining data and operations C. A collection of variables D. A compiler-level construct

Answer: B

3. ADT does NOT specify:

A. Type of data B. Set of operations C. Implementation details D. Behavior of operations

Answer: C

4. Which principle allows changing implementation without affecting usage?

A. Encapsulation B. Abstraction C. Modularity D. Inheritance

Answer: B

5. Which benefit of ADT MOST directly supports large software systems?

A. Faster execution B. Reduced memory usage C. Maintainability D. Hardware independence

Answer: C

6. Reusability in ADT means:

A. Same memory reused B. Same code reused without modification C. Same logical structure usable in different contexts D. Same algorithm reused for all problems

Answer: C

7. Which change does NOT violate ADT usage?

A. Changing stack from array to linked list B. Changing push operation behavior C. Removing pop operation D. Changing LIFO rule

Answer: A

8. Which of the following is NOT an ADT?

A. Stack B. Queue C. Array D. List

Answer: C (*Array is a concrete data structure, not an ADT*)

9. Stack ADT follows which access principle?

A. FIFO B. LIFO C. Random D. Priority-based

Answer: B

10. Queue ADT is MOST suitable for:

A. Function calls B. Expression evaluation C. Scheduling tasks D. Backtracking

Answer: C

11. Which ADT is hierarchical and non-linear?

A. Stack B. Queue C. List D. Tree

Answer: D

12. List ADT differs from Stack ADT because:

A. List allows insertion at any position B. Stack is dynamic C. List is non-linear D. Stack allows random access

Answer: A

13. Which statement is TRUE about ADTs?

A. ADTs are language dependent B. ADTs improve modularity C. ADTs increase memory wastage D. ADTs define memory layout

Answer: B

14. An array is BEST defined as:

A. A collection of heterogeneous elements B. A dynamic data structure C. A fixed-size collection stored contiguously D. A linked data structure

Answer: C

15. The main reason arrays support $O(1)$ access is:

A. Fixed size B. Same data type C. Contiguous memory allocation D. Compiler optimization

Answer: C

16. Address calculation in arrays depends on:

A. Index and base address B. Value stored

C. Array length only D. CPU speed

Answer: A

17. Which array property causes memory wastage?

A. Indexing B. Contiguous allocation C. Fixed size D. Fast access

Answer: C

18. Arrays are MOST suitable when:

A. Size varies frequently B. Frequent insertions/deletions are required C. Random access is required D. Data size is unknown

Answer: C

19. A two-dimensional array is conceptually represented as:

A. A list B. A matrix C. A tree D. A graph

Answer: B

20. Multi-dimensional arrays are MOST commonly used in:

A. Web development B. System programming C. Scientific computing D. Text processing

Answer: C

21. Which array type uses more complex address calculation?

A. One-dimensional B. Two-dimensional C. Both same D. Neither

Answer: B

22. Time complexity of array traversal is:

A. $O(1)$ B. $O(\log n)$ C. $O(n)$ D. $O(n^2)$

Answer: C

23. Worst-case time complexity of insertion at the beginning of an array is:

A. $O(1)$ B. $O(\log n)$ C. $O(n)$ D. $O(n^2)$

Answer: C

24. Deleting an element from the middle of an array requires:

A. Only deletion B. Memory reallocation C. Shifting elements D. Rehashing

Answer: C

25. Binary search on an array requires the array to be:

A. Small B. Fixed size C. Sorted D. Dynamic

Answer: C

26. Updating an array element at known index has time complexity:

A. $O(n)$ B. $O(\log n)$ C. $O(1)$ D. $O(n \log n)$

Answer: C

27. Which is NOT an advantage of arrays?

A. Fast access B. Easy implementation C. Dynamic resizing D. Cache friendliness

Answer: C

28. The MOST significant drawback of arrays is:

A. Slow traversal B. Fixed size C. Complex syntax D. High overhead

Answer: B

29. Frequent insertion and deletion in arrays is inefficient due to:

A. Indexing cost B. Memory fragmentation C. Element shifting D. Type conversion

Answer: C

30. Which structure overcomes array size limitation?

A. Stack ADT B. Queue ADT C. Linked list D. Tree ADT

Answer: C

31. Which statement is TRUE?

A. ADT defines memory layout B. Array is an ADT C. ADT hides implementation D. Array improves abstraction

Answer: C

32. ADT and Data Structure differ mainly in:

A. Syntax B. Performance

C. Level of abstraction D. Programming language

Answer: C

33. Stack ADT implemented using array will:

A. Lose LIFO behavior B. Gain dynamic size automatically C. Retain same operations D. Change access order

Answer: C

34. Which choice BEST represents abstraction?

A. `int arr[10]` B. Stack implemented using array C. Stack ADT with push/pop D. Memory address calculation

Answer: C

UNIT 6 & UNIT 7

IMPORTANT HIGH-LEVEL QUESTIONS FOR CCEE (CDAC)

1. Stack follows which principle?

A. FIFO B. LIFO C. Random access D. Priority-based

Answer: B

2. In a stack, insertion and deletion occur:

A. At different ends B. At the middle C. At the same end D. At both ends

Answer: C

3. The pointer used to manage a stack is:

A. Front B. Rear C. Top D. Head

Answer: C

4. Stack overflow occurs when:

A. Pop is attempted on empty stack B. Push is attempted on full stack C. Stack size becomes zero D. Stack becomes circular

Answer: B

5. Stack underflow occurs when:

A. Push on full stack B. Pop on empty stack C. Stack pointer is reset D. Stack size increases

Answer: B

6. Time complexity of push operation in stack is:

A. $O(n)$ B. $O(\log n)$ C. $O(1)$ D. $O(n^2)$

Answer: C

7. Peek operation in stack:

A. Removes element B. Returns bottom element C. Returns top element without removal D. Clears stack

Answer: C

8. Which stack operation does NOT modify stack content?

A. Push B. Pop C. Peek D. IsFull

Answer: C

9. Stack implemented using array suffers from:

A. Underflow only B. Overflow only C. Fixed size limitation D. Poor cache performance

Answer: C

10. Stack implemented using linked list overcomes:

A. Underflow B. Fixed size limitation C. LIFO behavior D. Function call overhead

Answer: B

11. Which data structure is used for function call management?

A. Queue B. Stack
C. Array D. Tree

Answer: B

12. Parenthesis checking algorithm uses:

A. Queue B. Stack C. Tree D. Graph

Answer: B

13. Undo/Redo functionality uses:

A. One stack B. Two stacks C. Queue D. Deque

Answer: B

14. Recursion uses stack mainly to store:

A. Input values B. Return addresses and local variables C. Global variables D. Heap memory

Answer: B

15. Infix to postfix conversion uses:

A. Queue B. Stack C. Linked list D. Deque

Answer: B

16. Queue follows which principle?

A. LIFO B. FIFO C. FILO D. Random

Answer: B

17. In a queue, insertion is done at:

A. Front B. Middle C. Rear D. Both ends

Answer: C

18. Deletion in a queue occurs at:

A. Rear B. Middle C. Front D. Both ends

Answer: C

19. Queue uses how many pointers?

A. One B. Two C. Three D. Depends on implementation

Answer: B

20. Queue overflow occurs when:

A. Dequeue on empty queue B. Enqueue on full queue C. Front equals rear D. Queue becomes circular

Answer: B

21. Simple (linear) queue suffers from:

A. Overflow B. Memory wastage C. Slow access D. Infinite loop

Answer: B

22. Circular queue solves:

A. Overflow B. Underflow C. Memory wastage D. Priority inversion

Answer: C

23. Circular queue uses which logic for wrapping?

A. Recursion B. Pointer arithmetic C. Modulo arithmetic D. Hashing

Answer: C

24. Priority queue serves elements based on:

A. Arrival time B. Insertion order C. Priority D. Stack order

Answer: C

25. Deque allows insertion and deletion:

A. Only at front B. Only at rear C. At both ends D. At middle only

Answer: C

SECTION F: QUEUE APPLICATIONS (VERY IMPORTANT)

26. CPU scheduling primarily uses:

A. Stack B. Queue C. Tree D. Array

Answer: B

27. Printer job management uses:

A. Stack B. Queue

C. Deque D. Tree

Answer: B

28. Breadth-First Search (BFS) uses:

A. Stack B. Queue C. Recursion D. Tree

Answer: B

29. Which traversal uses queue?

A. DFS B. BFS C. Inorder D. Postorder

Answer: B

30. Which statement is TRUE?

A. Stack uses FIFO B. Queue uses LIFO C. Stack uses one end D. Queue deletes from rear

Answer: C

31. Which structure is best for recursion?

A. Queue B. Stack C. Deque D. Array

Answer: B

32. Which structure is best for scheduling?

A. Stack B. Queue C. Tree D. Graph

Answer: B

33. Stack differs from queue mainly in:

A. Memory usage B. Access order C. Time complexity D. Implementation language

Answer: B

34. Circular queue differs from simple queue because:

A. Uses stack internally B. Has dynamic size C. Reuses freed space D. Uses recursion

Answer: C

UNIT 8: LINKED LIST UNIT 9: RECURSION

IMPORTANT MCQs (CCEE-FOCUSED)

1. A linked list differs from an array mainly because:

A. Linked list stores elements in sorted order B. Linked list uses contiguous memory C. Linked list uses non-contiguous memory D. Linked list supports random access

Answer: C

2. Each node of a singly linked list contains:

A. Data only B. Link only C. Data and link to next node D. Data, link to previous and next node

Answer: C

3. Accessing the 5th element in a linked list takes:

A. $O(1)$ B. $O(\log n)$ C. $O(n)$ D. $O(n \log n)$

Answer: C

4. Which operation is MOST efficient in a linked list compared to array?

A. Traversal B. Random access C. Insertion at beginning D. Binary search

Answer: C

5. Time complexity of inserting a node at the beginning of a linked list is:

A. $O(n)$ B. $O(\log n)$ C. $O(1)$ D. $O(n^2)$

Answer: C

6. Insertion at the end of a singly linked list without tail pointer takes:

A. $O(1)$ B. $O(\log n)$ C. $O(n)$ D. $O(n^2)$

Answer: C

7. Which linked list allows traversal in both directions?

A. Singly linked list B. Circular linked list C. Doubly linked list D. Linear list

Answer: C

8. The main drawback of doubly linked list is:

A. Slow insertion B. Extra memory usage C. Fixed size D. Sequential access

Answer: B

9. In a circular linked list:

A. First node points to NULL B. Last node points to NULL C. Last node points to first node D. First node points to itself

Answer: C

10. Which situation BEST suits a circular linked list?

A. Random access B. CPU scheduling C. Binary search D. Stack implementation

Answer: B

11. Searching in a linked list has time complexity:

A. $O(1)$ B. $O(\log n)$

C. $O(n)$ D. $O(n^2)$

Answer: C

12. Which statement is TRUE?

A. Linked list supports binary search B. Linked list wastes memory due to fixed size C. Linked list insertion does not require shifting D. Linked list provides $O(1)$ access

Answer: C

13. Which is NOT a limitation of arrays?

A. Fixed size B. Costly insertion C. Fast random access D. Memory wastage

Answer: C

14. Linked list nodes are connected using:

A. Index B. Array offset C. Pointer/reference D. Hash key

Answer: C

15. Which data structure is internally used to implement linked list nodes?

A. Array B. Structure / class C. Stack D. Queue

Answer: B

16. Recursion is a technique where:

A. A function executes repeatedly B. A function calls another function C. A function calls itself D. A loop calls a function

Answer: C

17. Which component is mandatory to stop recursion?

A. Loop condition B. Return statement C. Base condition D. Recursive call

Answer: C

18. Absence of base condition leads to:

A. Compilation error B. Logical error C. Stack overflow D. Memory leak

Answer: C

19. Which memory structure is used to manage recursive calls?

A. Heap B. Queue C. Stack D. Array

Answer: C

20. Each recursive call creates:

A. New heap memory B. New activation record C. New global variable D. New thread

Answer: B

21. Activation record contains:

A. Only function parameters B. Only return value C. Local variables and return address D. Global variables

Answer: C

22. Which recursion directly calls itself?

A. Indirect recursion B. Mutual recursion C. Tail recursion D. Direct recursion

Answer: D

23. Indirect recursion means:

A. Function calls itself at end B. Function calls itself through another function C. Function has multiple recursive calls D. Recursive call inside loop

Answer: B

24. Tail recursion is characterized by:

A. Recursive call at beginning B. Multiple recursive calls C. Recursive call as last statement D. No base condition

Answer: C

25. Tail recursion is important because:

A. Uses heap memory B. Uses less CPU C. Can be optimized by compiler D. Eliminates recursion

Answer: C

26. Which problem is MOST naturally solved using recursion?

A. Linear search B. Binary search C. Tree traversal D. Array traversal

Answer: C

27. Recursive solutions generally have:

A. Lower memory usage B. Faster execution
C. Cleaner and shorter code D. No overhead

Answer: C

28. Main disadvantage of recursion is:

A. Complex syntax B. Higher memory usage C. Poor readability D. No modularity

Answer: B

29. Recursive function for factorial of n uses stack depth of:

A. $O(1)$ B. $O(\log n)$ C. $O(n)$ D. $O(n^2)$

Answer: C

30. Which statement is TRUE?

A. Recursion never uses stack B. Iteration always slower than recursion C. Recursion may cause stack overflow D. Recursion cannot be converted to iteration

Answer: C

31. Recursion is internally implemented using:

A. Queue B. Stack C. Linked list D. Tree

Answer: B

32. Which is BETTER for memory efficiency?

A. Recursion B. Iteration C. Both same D. Depends on compiler

Answer: B

33. Which is TRUE for recursion vs iteration?

A. Recursion is faster B. Iteration uses stack C. Recursion uses stack memory D. Iteration causes overflow

Answer: C

B-TREE & B+ TREE

IMPORTANT MCQs FOR CCEE (CDAC PG-DAC)

1. B-Tree is mainly designed to:

A. Improve recursion B. Reduce disk I/O operations C. Replace BST D. Store data in RAM

Answer: B

2. B-Tree is a:

A. Binary tree B. Self-balancing binary tree C. Multi-way balanced search tree D. Unbalanced tree

Answer: C

3. B-Tree is MOST useful when data is stored in:

A. CPU registers B. Cache memory C. Main memory D. Secondary storage (disk)

Answer: D

4. Which tree minimizes height to reduce disk access?

A. AVL tree B. BST C. B-Tree D. Heap

Answer: C

5. In a B-Tree, all leaf nodes are at:

A. Different levels B. Random levels C. Same level D. Root level

Answer: C

6. If order of B-Tree is m, maximum children per node is:

A. $m - 1$ B. m C. $2m$ D. $m + 1$

Answer: B

7. Maximum number of keys in a B-Tree node of order m is:

A. m B. $m + 1$ C. $m - 1$ D. $\lceil m/2 \rceil$

Answer: C

8. Minimum number of children (except root) in B-Tree of order m is:

A. 1 B. m C. $\lceil m/2 \rceil$ D. $m - 1$

Answer: C

9. Minimum number of keys (except root) in B-Tree of order m is:

A. $\lceil m/2 \rceil$ B. $\lceil m/2 \rceil - 1$ C. $m - 1$ D. 1

Answer: B

10. Root of a B-Tree:

A. Must have m children B. Must have $\lceil m/2 \rceil$ children C. Can have fewer keys than other nodes D. Cannot be a leaf

Answer: C

11. Time complexity of search in B-Tree is:

A. $O(n)$ B. $O(\log n)$
C. $O(n \log n)$ D. $O(1)$

Answer: B

12. Insertion in B-Tree may cause:

A. Rotation B. Node splitting C. Node deletion D. Tree skewing

Answer: B

13. When a B-Tree node overflows:

A. It is deleted B. It is ignored C. It is split and middle key is promoted D. Tree becomes unbalanced

Answer: C

14. Deletion in B-Tree may require:

A. Rotation only B. Merging or borrowing C. Rebuilding entire tree D. No adjustment

Answer: B

15. Height of B-Tree grows as:

A. Linear B. Exponential C. Logarithmic D. Constant

Answer: C

16. B+ Tree differs from B-Tree because:

A. It is not balanced B. It stores data only in internal nodes C. It stores data only in leaf nodes D. It allows duplicate keys

Answer: C

17. In B+ Tree, internal nodes store:

A. Data only B. Keys and data C. Keys only D. Pointers only

Answer: C

18. Leaf nodes in B+ Tree are:

A. Isolated B. Connected randomly C. Linked sequentially D. Connected to root

Answer: C

19. Major advantage of linked leaf nodes in B+ Tree is:

A. Faster insertion B. Faster deletion C. Efficient range queries D. Reduced height

Answer: C

20. Which tree is MOST preferred for database indexing?

A. BST B. AVL Tree C. B-Tree D. B+ Tree

Answer: D

21. Which statement is TRUE?

A. B-Tree stores data only in leaves B. B+ Tree stores data in all nodes C. B+ Tree stores data only in leaves D. B-Tree has linked leaves

Answer: C

22. Range queries are faster in:

A. BST B. AVL C. B-Tree D. B+ Tree

Answer: D

23. Internal nodes in B+ Tree are used mainly as:

A. Data storage B. Indexing structure C. Leaf connectors D. Memory blocks

Answer: B

24. Which tree provides uniform access time for all data records?

A. BST B. AVL Tree C. B-Tree D. B+ Tree

Answer: D

25. B+ Tree improves disk efficiency because:

A. More keys fit in internal nodes B. Less memory is used C. Data is duplicated D. Tree height increases

Answer: A

26. B-Tree is NOT:

A. Balanced B. Multi-way C. Binary D. Logarithmic height

Answer: C

27. Which tree is most suitable for secondary storage?

A. AVL B. BST

C. Heap D. B+ Tree

Answer: D

28. Compared to B-Tree, B+ Tree:

A. Has greater height B. Has smaller height C. Has same height but faster search D. Is unbalanced

Answer: C

29. In B+ Tree, search always ends at:

A. Root B. Any internal node C. Leaf node D. Middle level

Answer: C

30. Which is TRUE for both B-Tree and B+ Tree?

A. Binary structure B. Balanced height C. Data only in leaves D. Unordered keys

Answer: B

31. B-Tree reduces disk I/O mainly by:

A. Using recursion B. Increasing branching factor C. Using hashing D. Reducing key size

Answer: B

32. Why are B-Trees called multi-way trees?

A. Multiple roots B. Multiple parents C. More than two children per node D. Multiple levels

Answer: C

33. Which tree supports sequential access efficiently?

A. AVL B. BST C. B-Tree D. B+ Tree

Answer: D

34. B-Tree insertion ensures balance by:

A. Rotation B. Rehashing C. Splitting nodes D. Sorting nodes

Answer: C

35. In CCEE, B-Tree/B+ Tree questions MOSTLY test:

A. Pointer syntax B. Memory addresses C. Conceptual differences & applications D. Code tracing

Answer: C

UNIT 11 & UNIT 12

IMPORTANT MCQs FOR CCEE (CDAC PG-DAC)

1. Linear search works by:

A. Dividing the array into halves B. Comparing all elements simultaneously C. Sequentially checking each element D. Hashing elements

Answer: C

2. Linear search is MOST suitable when:

A. Data is very large B. Data is sorted C. Data is unsorted and small D. Random access is not allowed

Answer: C

3. Worst-case time complexity of linear search is:

A. $O(1)$ B. $O(\log n)$ C. $O(n)$ D. $O(n \log n)$

Answer: C

4. Best-case time complexity of linear search is:

A. $O(n)$ B. $O(\log n)$ C. $O(1)$ D. $O(n^2)$

Answer: C

5. Binary search requires the data to be:

A. Dynamic B. Small C. Sorted D. Unique

Answer: C

6. Binary search follows which approach?

A. Greedy B. Dynamic programming C. Divide and conquer D. Backtracking

Answer: C

7. Time complexity of binary search in worst case is:

A. $O(1)$ B. $O(n)$ C. $O(\log n)$ D. $O(n \log n)$

Answer: C

8. Binary search is NOT suitable for:

A. Sorted arrays B. Arrays with random access C. Linked lists D. Large datasets

Answer: C

9. Recursive binary search uses stack space of:

A. $O(1)$ B. $O(\log n)$ C. $O(n)$ D. $O(n \log n)$

Answer: B

10. Compared to linear search, binary search is:

A. Slower B. More complex but faster C. Always slower D. Only theoretical

Answer: B

11. Sorting means:

A. Searching elements B. Arranging elements in a specific order

C. Removing duplicates D. Compressing data

Answer: B

12. Which of the following sorting algorithms is comparison-based?

A. Counting sort B. Radix sort C. Merge sort D. Bucket sort

Answer: C

13. Stable sorting algorithm means:



A. Sorting without recursion B. Sorting with minimum swaps C. Preserving relative order of equal elements D. Sorting in place

Answer: C

14. Bubble sort works by:

A. Selecting minimum element B. Inserting elements in correct position C. Repeatedly swapping adjacent elements D. Dividing array

Answer: C

15. Worst-case time complexity of bubble sort is:

A. $O(n)$ B. $O(\log n)$ C. $O(n^2)$ D. $O(n \log n)$

Answer: C

16. Which sorting algorithm always takes $O(n^2)$ time?

A. Bubble sort B. Insertion sort C. Selection sort D. Quick sort

Answer: C

17. Selection sort selects:

A. Maximum element repeatedly B. Minimum element repeatedly C. Middle element repeatedly D. Random element repeatedly

Answer: B

18. Insertion sort is MOST efficient when:

A. Data is large B. Data is reverse sorted C. Data is nearly sorted D. Data is random

Answer: C

19. Best-case time complexity of insertion sort is:

A. $O(n^2)$ B. $O(n \log n)$ C. $O(n)$ D. $O(\log n)$

Answer: C

20. Which of the following is TRUE?

A. Bubble sort is fastest B. Selection sort is stable C. Insertion sort is stable D. Selection sort is adaptive

Answer: C

21. Merge sort is based on:

A. Greedy method B. Divide and conquer C. Backtracking D. Brute force

Answer: B

22. Time complexity of merge sort in worst case is:

A. $O(n^2)$ B. $O(n \log n)$ C. $O(\log n)$ D. $O(n)$

Answer: B

23. Merge sort requires extra space because:

A. It uses recursion B. It uses linked lists C. It uses temporary arrays during merging D. It swaps elements

Answer: C

24. Merge sort is preferred when:

A. Memory is limited B. Stability is required C. Data is small D. Data is already sorted

Answer: B

25. Merge sort is:

A. In-place B. Not stable C. Stable D. Quadratic

Answer: C

26. Quick sort is based on:

A. Divide and conquer B. Dynamic programming C. Greedy D. Brute force

Answer: A

27. Worst-case time complexity of quick sort is:

A. $O(n \log n)$ B. $O(\log n)$

C. $O(n^2)$ D. $O(n)$

Answer: C

28. Quick sort becomes worst-case when:

A. Pivot is always middle B. Pivot is always smallest or largest C. Data is random D. Data is small

Answer: B

29. Which statement is TRUE about quick sort?

A. It is stable B. It always takes $O(n \log n)$ C. It is in-place D. It needs extra array

Answer: C

30. In practice, quick sort is:

A. Slowest B. Rarely used C. Very fast D. Avoided

Answer: C

31. Heap sort uses which data structure?

A. Stack B. Queue C. Heap D. Tree only

Answer: C

32. Time complexity of heap sort in worst case is:

A. $O(n^2)$ B. $O(n \log n)$ C. $O(\log n)$ D. $O(n)$

Answer: B

33. Heap sort is:

A. Stable B. Not stable C. Recursive D. Adaptive

Answer: B

34. Which is TRUE about heap sort?

A. Requires extra space B. Worst-case better than quick sort C. Slower than bubble sort D. Not comparison-based

Answer: B

35. Which sorting algorithm guarantees $O(n \log n)$ in all cases?

A. Quick sort B. Merge sort C. Heap sort D. Both B and C

Answer: D

36. Which algorithm is best for large datasets when memory is NOT a concern?

A. Bubble sort B. Insertion sort C. Merge sort D. Selection sort

Answer: C

37. Which algorithm is best for nearly sorted data?

A. Quick sort B. Heap sort C. Insertion sort D. Selection sort

Answer: C

38. Binary search is faster than linear search because:

A. It uses recursion B. It reduces search space by half C. It uses stack D. It uses hashing

Answer: B

39. Sorting before binary search:

A. Is unnecessary B. Increases overall complexity C. Makes binary search invalid D. Always faster

Answer: B

40. In CCEE, sorting questions MOSTLY test:

A. Code syntax B. Memory addresses C. Time complexity & behavior D. Pointer arithmetic

Answer: C

UNIT 13: HASHING

IMPORTANT MCQs FOR CCEE (CDAC PG-DAC)

1. Hashing is a technique used to:

A. Sort data efficiently B. Map keys directly to memory locations C. Reduce memory usage D. Traverse data structures

Answer: B

2. Hashing provides which time complexity for search in average case?

A. $O(n)$ B. $O(\log n)$ C. $O(1)$ D. $O(n \log n)$

Answer: C

3. The data structure used in hashing is:

A. Stack B. Queue C. Array D. Tree

Answer: C

4. Hashing is MOST suitable for:

A. Range queries B. Sorted data retrieval C. Fast exact-match searching D. Sequential processing

Answer: C

5. Hashing performance mainly depends on:

A. Size of key B. Quality of hash function C. Programming language D. CPU speed

Answer: B

6. A hash function should ideally:

A. Be complex B. Be slow C. Distribute keys uniformly D. Produce sorted indices

Answer: C

7. Division method hash function is:

A. $h(k) = k^2$ B. $h(k) = k / \text{table_size}$ C. $h(k) = k \% \text{table_size}$ D. $h(k) = \text{mid}(k)$

Answer: C

8. In division method, table size is preferably:

A. Even B. Power of 2 C. Prime number D. Multiple of key size

Answer: C

9. Mid-square method works by:

A. Dividing key B. Squaring key and extracting middle digits C. Folding digits D. Using two hash functions

Answer: B

10. Folding method is MOST suitable for:

A. Small keys B. Floating point numbers C. Large numeric keys D. Sorted keys

Answer: C

11. Collision occurs when:

A. Hash function fails B. Table is full
C. Two keys map to same index D. Key is too large

Answer: C

12. Collision in hashing is:

A. Always avoidable B. Always an error C. Unavoidable D. Only in linear probing

Answer: C

13. A good hash function aims to:

A. Eliminate collisions completely B. Minimize collisions C. Increase collisions D. Ignore collisions

Answer: B

14. Linear probing resolves collision by:

A. Using linked list B. Using second hash function C. Checking next available slot sequentially D. Rehashing entire table

Answer: C

15. Linear probing suffers from:

A. Secondary clustering B. Primary clustering C. No clustering D. Rehashing

Answer: B

16. Quadratic probing resolves collision using:

A. Linear increments B. Square increments C. Two hash functions D. Linked lists

Answer: B

17. Quadratic probing reduces:

A. Primary clustering B. Secondary clustering C. Both clusterings D. No clustering

Answer: A

18. Double hashing uses:

A. One hash function B. Three hash functions C. Two hash functions D. Recursion

Answer: C

19. Double hashing is preferred because it:

A. Is easiest B. Uses less memory C. Minimizes clustering D. Is fastest to compute

Answer: C

20. Which collision resolution technique gives BEST distribution?

A. Linear probing B. Quadratic probing C. Double hashing D. Chaining

Answer: C

SECTION E: HASH TABLE OPERATIONS

21. Average time complexity of insertion in hashing is:

A. $O(n)$ B. $O(\log n)$ C. $O(1)$ D. $O(n \log n)$

Answer: C

22. Worst-case time complexity of hashing operations is:

A. $O(1)$ B. $O(\log n)$ C. $O(n)$ D. $O(n^2)$

Answer: C

23. Search operation in open addressing must:

A. Scan entire table B. Use same probing sequence as insertion C. Use binary search D. Rehash table

Answer: B

24. Deletion in open addressing is difficult because:

A. Index is lost B. It breaks probe sequence C. Data is sorted D. Hash function changes

Answer: B

25. Lazy deletion means:

A. Immediate removal B. Rehashing table C. Marking slot as deleted D. Ignoring deletion

Answer: C

26. Hashing is NOT suitable for:

A. Exact match searching B. Symbol tables C. Range queries D. Caches

Answer: C

27. Which statement is TRUE?

A. Hashing guarantees $O(1)$ always B. Hashing is sorted structure

C. Hashing gives $O(1)$ average case D. Hashing avoids collisions

Answer: C

28. Primary clustering occurs in:

A. Quadratic probing B. Double hashing C. Linear probing D. Chaining

Answer: C

29. Secondary clustering occurs in:

A. Linear probing B. Quadratic probing C. Double hashing D. Chaining

Answer: B

30. Load factor in hashing represents:

A. Size of table B. Number of collisions C. Ratio of filled slots to table size D. Hash function strength

Answer: C

31. As load factor increases:

A. Performance improves B. Collisions decrease C. Performance degrades D. Table shrinks

Answer: C

32. Hashing is faster than binary search because:

A. It uses recursion B. It avoids comparisons C. It avoids sorting D. It uses pointers

Answer: B

33. Which is TRUE about hashing?

A. It preserves order B. It supports traversal C. It is not comparison-based D. It guarantees worst-case $O(1)$

Answer: C

34. Which application uses hashing?

A. Expression evaluation B. Symbol tables C. Tree traversal D. BFS

Answer: B

35. In CCEE, hashing questions MOSTLY test:

A. Pointer syntax B. Memory layout C. Conceptual understanding of collisions D. Language-specific code

Answer: C

UNIT 14: GRAPHS

COMPLETE & IMPORTANT MCQs FOR CCEE (CDAC)

1. A graph is defined as:

A. A linear collection of elements B. A hierarchical structure C. A set of vertices and edges D. A tree with cycles

Answer: C

2. In graph terminology, vertices are also called:

A. Edges B. Nodes C. Paths D. Links

Answer: B

3. An edge in a graph represents:

A. A vertex B. A weight C. A connection between vertices D. A traversal

Answer: C

4. A graph with no edges is called:

A. Complete graph B. Null graph C. Cyclic graph D. Directed graph

Answer: B

5. Maximum number of edges in an undirected graph with n vertices is:

A. n B. n^2 C. $n(n-1)/2$ D. $n-1$

Answer: C

6. In a directed graph:

A. Edges have no direction B. Edges have direction C. All edges are bidirectional D. Graph must be cyclic

Answer: B

7. An undirected graph differs from a directed graph because:

A. It has weights B. It has cycles C. Edges have no direction D. It has fewer vertices

Answer: C

8. A weighted graph is one in which:

A. Vertices have weights B. Edges have weights C. Graph has cycles D. Graph is directed

Answer: B

9. A graph containing at least one cycle is called:

A. Acyclic B. Tree C. Cyclic D. Complete

Answer: C

10. A Directed Acyclic Graph (DAG) is:

A. Undirected and cyclic B. Directed and cyclic C. Directed and acyclic D. Undirected and acyclic

Answer: C

11. Adjacency matrix representation requires space:

A. $O(V + E)$ B. $O(E^2)$

C. $O(V^2)$ D. $O(E)$

Answer: C

12. Adjacency list representation requires space:

A. $O(V^2)$ B. $O(V + E)$ C. $O(E^2)$ D. $O(V \log E)$

Answer: B

13. Adjacency matrix is MOST suitable for:

A. Sparse graphs B. Dense graphs C. Large sparse graphs D. Trees only

Answer: B

14. Adjacency list is MOST suitable for:

A. Dense graphs B. Complete graphs C. Sparse graphs D. Small graphs only

Answer: C

15. Checking existence of an edge is faster in:

A. Adjacency list B. Adjacency matrix C. DFS D. BFS

Answer: B

16. BFS traversal uses:

A. Stack B. Recursion C. Queue D. Priority queue

Answer: C

17. DFS traversal uses:

A. Queue B. Stack or recursion C. Heap D. Array

Answer: B

18. BFS explores nodes:

A. Depth-wise B. Randomly C. Level by level D. In sorted order

Answer: C

19. DFS explores nodes:

A. Level by level B. Breadth first C. As deep as possible D. In increasing order

Answer: C

20. Time complexity of BFS and DFS is:

A. $O(V)$ B. $O(E)$ C. $O(V + E)$ D. $O(V^2)$

Answer: C

21. BFS is preferred when:

A. Detecting cycles B. Finding shortest path in unweighted graph C. Topological sorting D. Checking connectivity

Answer: B

22. DFS is useful for:

A. Shortest path B. Level order traversal C. Cycle detection D. Priority scheduling

Answer: C

23. Dijkstra's algorithm finds:

A. All-pairs shortest paths B. Shortest path in unweighted graph C. Single-source shortest path D. Minimum spanning tree

Answer: C

24. Dijkstra's algorithm does NOT work with:

A. Directed graphs B. Undirected graphs C. Negative edge weights D. Weighted graphs

Answer: C

25. Dijkstra's algorithm is based on:

A. Dynamic programming B. Greedy approach C. Backtracking D. Divide and conquer

Answer: B

26. Time complexity of Dijkstra using adjacency matrix is:

A. $O(E \log V)$ B. $O(V^2)$ C. $O(V + E)$ D. $O(E^2)$

Answer: B

27. Floyd-Warshall algorithm finds:

A. Single-source shortest path B. Shortest path in trees

C. All-pairs shortest path D. Minimum spanning tree

Answer: C

28. Floyd-Warshall algorithm uses:

A. Greedy technique B. Divide and conquer C. Dynamic programming D. Backtracking

Answer: C

29. Time complexity of Floyd-Warshall algorithm is:

A. $O(V^2)$ B. $O(E \log V)$ C. $O(V^3)$ D. $O(V + E)$

Answer: C

30. Floyd–Warshall can handle:

A. Negative edges B. Negative cycles C. Only unweighted graphs D. Trees only

Answer: A

31. A spanning tree of a graph with V vertices has:

A. V edges B. $V - 1$ edges C. $V + 1$ edges D. V^2 edges

Answer: B

32. A spanning tree:

A. Contains cycles B. Is disconnected C. Connects all vertices without cycles D. Has maximum edges

Answer: C

33. A Minimum Spanning Tree (MST) is:

A. Any spanning tree B. Spanning tree with minimum weight C. Shortest path tree D. Directed tree

Answer: B

34. Prim's algorithm is:

A. Edge-based B. Vertex-based C. Dynamic programming D. Recursive only

Answer: B

35. Kruskal's algorithm is:

A. Vertex-based B. Node-based C. Edge-based D. Level-based

Answer: C

36. Kruskal's algorithm uses:

A. Stack B. Queue C. Union–Find (Disjoint Set) D. Heap only

Answer: C

37. Prim's algorithm is best suited for:

A. Sparse graphs B. Dense graphs C. Disconnected graphs D. Directed graphs

Answer: B

38. Kruskal's algorithm is best suited for:

A. Dense graphs B. Complete graphs C. Sparse graphs D. Directed graphs

Answer: C

39. Which algorithm avoids cycle formation explicitly?

A. BFS B. DFS C. Prim D. Kruskal

Answer: D

40. Which statement is TRUE?

A. BFS uses stack B. DFS uses queue C. Dijkstra allows negative edges D. MST contains no cycles

Answer: D

41. Which traversal guarantees shortest path in unweighted graph?

A. DFS B. BFS C. Dijkstra D. Kruskal

Answer: B

42. Which graph representation is memory efficient for sparse graphs?

A. Adjacency matrix B. Adjacency list C. Incidence matrix D. Edge list only

Answer: B

UNIT 15: ALGORITHM DESIGN TECHNIQUES

IMPORTANT MCQs FOR CCEE (CDAC PG-DAC)

1. Algorithm design techniques primarily help in:

A. Writing syntax-correct code B. Choosing data structures C. Deciding how to approach a problem D. Reducing memory only

Answer: C

2. Which of the following is NOT an algorithm design technique?

A. Divide and Conquer B. Greedy C. Hashing D. Dynamic Programming

Answer: C

3. Choosing the correct algorithm design technique mainly affects:

A. Code readability B. Problem solvability and efficiency C. Compiler optimization D. Hardware utilization

Answer: B

4. Divide and Conquer works by:

A. Trying all solutions B. Making greedy choices C. Breaking problem into independent subproblems D. Storing results of subproblems

Answer: C

5. Which of the following algorithms uses Divide and Conquer?

A. Dijkstra B. Prim C. Binary Search D. Floyd–Warshall

Answer: C

6. Merge Sort is an example of:

A. Greedy algorithm B. Dynamic Programming C. Divide and Conquer D. Backtracking

Answer: C

7. Divide and Conquer does NOT:

A. Use recursion B. Divide problem C. Store intermediate results D. Combine results

Answer: C

8. Time complexity improvement in Divide and Conquer is mainly due to:

A. Random choices B. Problem size reduction C. Avoiding recursion D. Avoiding comparisons

Answer: B

9. Greedy algorithms work by:

A. Exploring all possibilities B. Choosing locally optimal solutions C. Storing all subproblem results D. Random selection

Answer: B

10. Which property must a problem satisfy for greedy solution to be optimal?

A. Optimal substructure only B. Overlapping subproblems C. Greedy choice property D. Recursion

Answer: C

11. Which of the following is a greedy algorithm?

A. Binary Search B. Merge Sort

C. Dijkstra's Algorithm D. Floyd-Warshall

Answer: C

12. Greedy algorithms:

A. Always give optimal solution B. Never give optimal solution C. Give optimal solution only for certain problems D. Require backtracking

Answer: C

13. Prim's and Kruskal's algorithms are used for:

A. Shortest path B. Sorting C. Minimum Spanning Tree D. Searching

Answer: C

14. Greedy algorithms generally:

A. Reconsider previous decisions B. Use memoization C. Do not revise past choices D. Use dynamic tables

Answer: C

15. Dynamic Programming is mainly used when:

A. Subproblems are independent B. Subproblems overlap C. Only recursion is needed D. Input size is small

Answer: B

16. Which two properties are required for Dynamic Programming?

A. Recursion and iteration B. Greedy choice and sorting C. Optimal substructure and overlapping subproblems D. Stack and queue

Answer: C

17. Which algorithm uses Dynamic Programming?

A. Quick Sort B. Binary Search C. Floyd-Warshall D. Dijkstra

Answer: C

18. Dynamic Programming improves efficiency by:

A. Using recursion B. Eliminating recursion C. Storing subproblem results D. Using greedy decisions

Answer: C

19. Memoization refers to:

A. Bottom-up approach B. Iterative solution C. Top-down DP with caching D. Greedy optimization

Answer: C

20. Tabulation refers to:

A. Recursive DP B. Bottom-up DP C. Greedy method D. Divide and Conquer

Answer: B

21. Which statement is TRUE?

A. Divide & Conquer stores results B. Dynamic Programming avoids recursion C. DP avoids recomputation D. Greedy always revisits choices

Answer: C

22. Brute force algorithm:

A. Uses heuristic pruning B. Uses optimal substructure C. Tries all possible solutions D. Uses randomization

Answer: C

23. Brute force approach is BEST suited when:

A. Input size is large B. Constraints are complex C. Input size is very small D. Optimization is required

Answer: C

24. Which algorithm is a brute force technique?

A. Binary search B. Linear search C. Merge sort D. Quick sort

Answer: B

25. Main disadvantage of brute force is:

A. Complex implementation B. High memory usage C. Exponential time complexity D. Inaccuracy

Answer: C

26. Backtracking is used to solve:

A. Optimization problems only B. Constraint satisfaction problems C. Sorting problems D. Searching problems

Answer: B

27. Backtracking differs from brute force because it:

A. Tries fewer possibilities B. Uses randomness
C. Stops exploring invalid paths early D. Uses dynamic programming

Answer: C

28. Which problem is solved using backtracking?

A. Dijkstra B. Merge Sort C. N-Queen D. Binary Search

Answer: C

29. Backtracking algorithm follows:

A. BFS B. DFS C. Greedy approach D. Dynamic programming

Answer: B

30. Backtracking involves:

A. Try → Accept → Store B. Try → Fail → Undo C. Split → Merge D. Select → Lock

Answer: B

31. Branch and Bound is mainly used for:

A. Decision problems B. Searching problems C. Optimization problems D. Sorting problems

Answer: C

32. Branch and Bound differs from backtracking because it:

A. Uses recursion B. Uses cost bounds to prune C. Tries all solutions D. Avoids DFS

Answer: B

33. Which problem commonly uses Branch and Bound?

A. Linear search B. Binary search C. Traveling Salesman Problem D. BFS

Answer: C

34. Branch and Bound prunes branches based on:

A. Validity B. Cost estimation C. Depth D. Random choice

Answer: B

35. Stochastic algorithms use:

A. Deterministic rules B. Dynamic tables C. Randomized decisions D. Greedy decisions

Answer: C

36. Output of stochastic algorithms:

A. Is always same B. Is always optimal C. May vary between executions D. Is always incorrect

Answer: C

37. Which algorithm uses randomization?

A. Selection sort B. Randomized Quick Sort C. Merge Sort D. Floyd–Warshall

Answer: B

38. Stochastic algorithms are useful when:

A. Exact solution is mandatory B. Search space is very large C. Data is sorted D. Problem is simple

Answer: B

39. Which technique stores subproblem results?

A. Divide and Conquer B. Greedy C. Dynamic Programming D. Backtracking

Answer: C

40. Which technique always tries all possibilities?

A. Greedy B. Brute Force C. Branch and Bound D. Dynamic Programming

Answer: B

41. Which technique is BEST for constraint satisfaction?

A. Greedy B. Backtracking C. Divide and Conquer D. Stochastic

Answer: B

42. Which technique guarantees optimal solution when applicable?

A. Greedy B. Dynamic Programming C. Brute Force D. Stochastic

Answer: B

43. In CCEE, algorithm design questions MOSTLY test:

A. Code syntax B. Memory addresses
C. Correct technique identification D. Language features

Answer: C

